

# Algorithm for file updates in Python

## Project description

In this project, I was controlling access to restricted content. IP addresses that were no longer allowed to access the restricted content were being removed to minimise attack surface. All this information was on a system log file. The “allow\_list.txt” file is the one im using and removing ip\_addresses no longer allowed.

## Open the file that contains the allow list

```
In [3]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

A line of code “with open(import\_file, “r”) as file:” simply runs the open file code that takes two parameters where the filename is placed as first parameter and then the action to take on that file is placed as the second parameter. The “r” reads the data on the file and names the filename as file for future use. This is one of the built-in python functions.

## Read the file contents

```
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)
```

ip\_address 192.168.205.12 192.168.6.9 192.168.52.90 192.168.90.124 192.168.186.176 192.168.133.188 192.168.218.219  
192.168.52.37 192.168.156.224 192.168.60.153 192.168.69.116

The .read() function is used to read the imported file “file” and the read data stored in a variable named ip\_addresses “ip\_addresses = file.read()”, then results displayed on the screen using the print(ip\_addresses) function to print/display results to screen.

## Convert the string into a list

```
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

    # Use `.split()` to convert `ip_addresses` from a string to a list
    ip_addresses = ip_addresses.split() ✓

    # Display `ip_addresses`
    print(ip_addresses) ✓
```

['ip\_address', '192.168.25.60', '192.168.205.12', '192.168.97.225', '192.168.6.9', '192.168.52.90', '192.168.158.170', '192.168.90.124', '192.168.186.176', '192.168.133.188', '192.168.203.198', '192.168.201.40', '192.168.218.219', '192.168.52.37', '192.168.156.224', '192.168.60.153', '192.168.58.57', '192.168.69.116']

.split() function is used to convert a string to a list. This is possible when the ip\_addresses string is changed using the .split() function. This function changes the string to list through ip\_addresses = ip\_addresses.split() method.

## Iterate through the remove list

```
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named ip_addresses
    ip_addresses = file.read()

    # Use `.split()` to convert `ip_addresses` from a string to a list

    ip_addresses = ip_addresses.split()

    # Build iterative statement
    # Name loop variable `element`
    # Loop through `ip_addresses`

    for element in ip_addresses:
        if element in remove_list:
```

6

```
    # Display `element` in every iteration
    print(element)
```

```
192.168.97.225
192.168.158.170
192.168.201.40
192.168.58.57
```

Introduction of a “if” conditional statement and a “for” loop statement made it possible to iterate through the given lists of the `ip_addresses` and also `remove_list` to see if any ip addresses that are still in `ip_addresses` appear on the list of `remove_list` of ip addresses that are no longer allowed to access the system. 4 different ip addresses are found that should not be allowed to access the system and are printed to the screen through the `print` function. All those addresses are stored in a variable called `element`.

## Remove IP addresses that are on the remove list

```
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Display `ip_addresses`

print(ip_addresses)
```

```
['ip_address', '192.168.25.60', '192.168.205.12', '192.168.6.9',
'192.168.52.90', '192.168.90.124', '192.168.186.176', '192.168.133.188',
'192.168.203.198', '192.168.218.219', '192.168.52.37', '192.168.156.224',
'192.168.60.153', '192.168.69.116']
```

Hint 1

The ip addresses in the remove\_list are removed using the .remove() function. We use ip\_addresses.remove(element) method where element variable is passed as a parameter to the .remove() function to show that we are removing element found on the remove\_list from the ip\_addresses list. When we display the list of addresses the remove\_list ip addresses are no longer there.

## Update the file with the revised list of IP addresses

```
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named ip_addresses
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)
```

```
# Convert `ip_addresses` back to a string so that it can be written into the  
→ text file  
  
ip_addresses = " ".join(ip_addresses)  
  
# Build `with` statement to rewrite the original file  
  
with open(import_file, "w") as file:  
  
    # Rewrite the file, replacing its contents with `ip_addresses`  
  
    file.write(ip_addresses)
```

The `ip_addresses = " ".join(ip_addresses)` is used to convert the list back to a string of ip addresses then results written back to the original file using `"with open(import_file, "w") as file:"` method. Using the `file.write(ip_addresses)` writes the new information to the old file.

## Summary

Algorithms can be efficiently developed using a combination of functions, conditional statements, and variables. Importing files and using methods such as `.read()` and `.write()` allows log files to be updated and managed effectively. Information such as IP addresses can be removed from logs when access is no longer allowed by using the `.remove()` method and updating the file accordingly. Additionally, functions enable efficient information retrieval by allowing relevant data to be accessed through the use of arguments.